

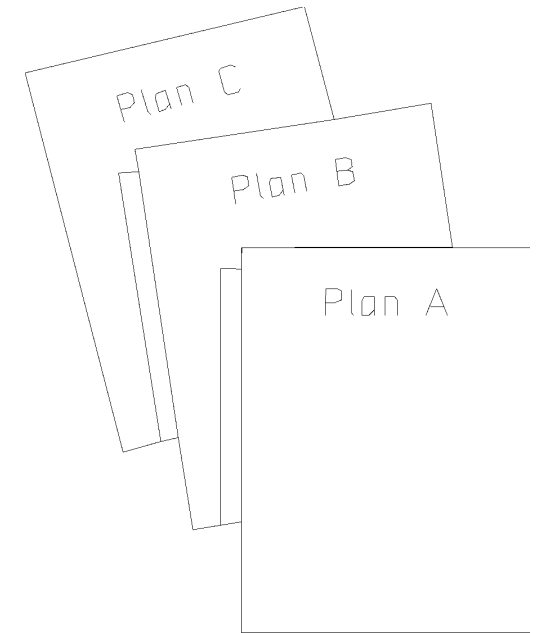
Computer System Basics

서울대학교 컴퓨터공학부 이영기



Overview

- Objective
 - To understand what matters for efficient execution for ML
- Content
 - Why systems matter for ML?
 - Memory hierarchy and data movement
 - Tensor computation as data reuse
 - Parallel execution models



Today's Agenda

- Why systems matter for ML?
- Memory hierarchy and data movement
- Tensor computation as data reuse
- Parallel execution models

FLOPs Are Not Latency

하나의 NN 을 추론하는데 드는 연산의 개수 /
floating point 계산량 per second

- FLOPs measure only arithmetic work, not the full execution cost.
- Actual latency includes data movement, runtime overhead, etc.
- Reducing FLOPs does not guarantee proportional latency reduction.

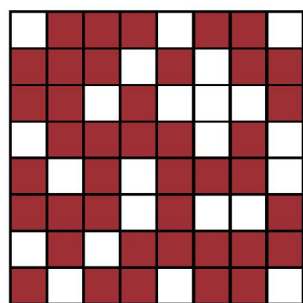
**Actual Latency = f (overhead,
data movement,
arithmetic)**

Model Optimization != System Speedup

- Pruning, quantization, and distillation reduce model-level cost.
- Unstructured sparsity may reduce FLOPs but introduce irregular access.
- Quantization may reduce traffic, but can slow down (e.g., K-means quantization).

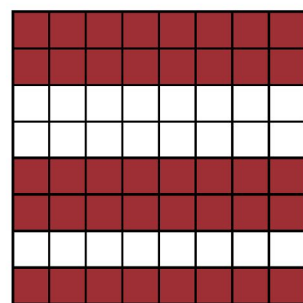
Pruning at Different Granularities

- Pruning is performed at different granularities, from structured to unstructured.



Fine-grained/Unstructured

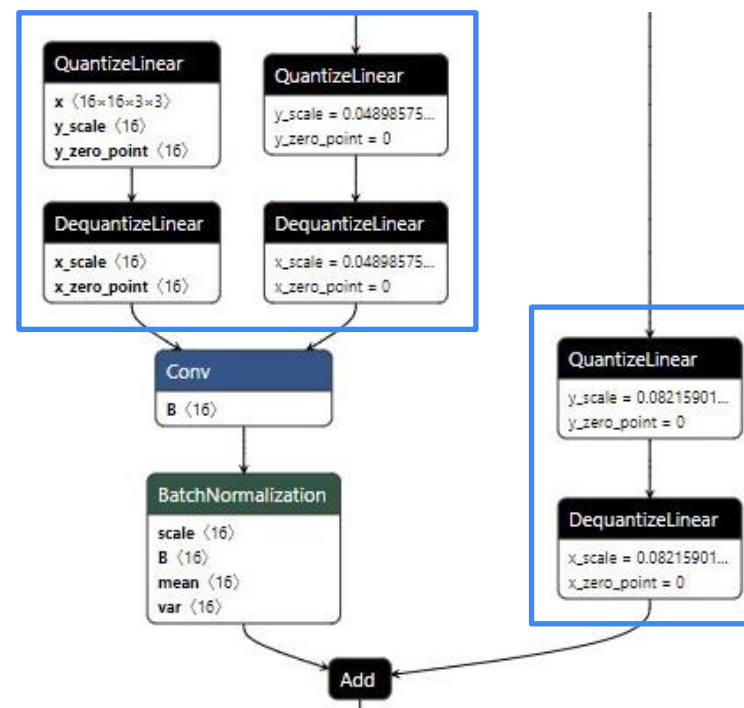
- More flexible pruning index choice
- Hard to accelerate (irregular data expression)



Coarse-grained/Structured

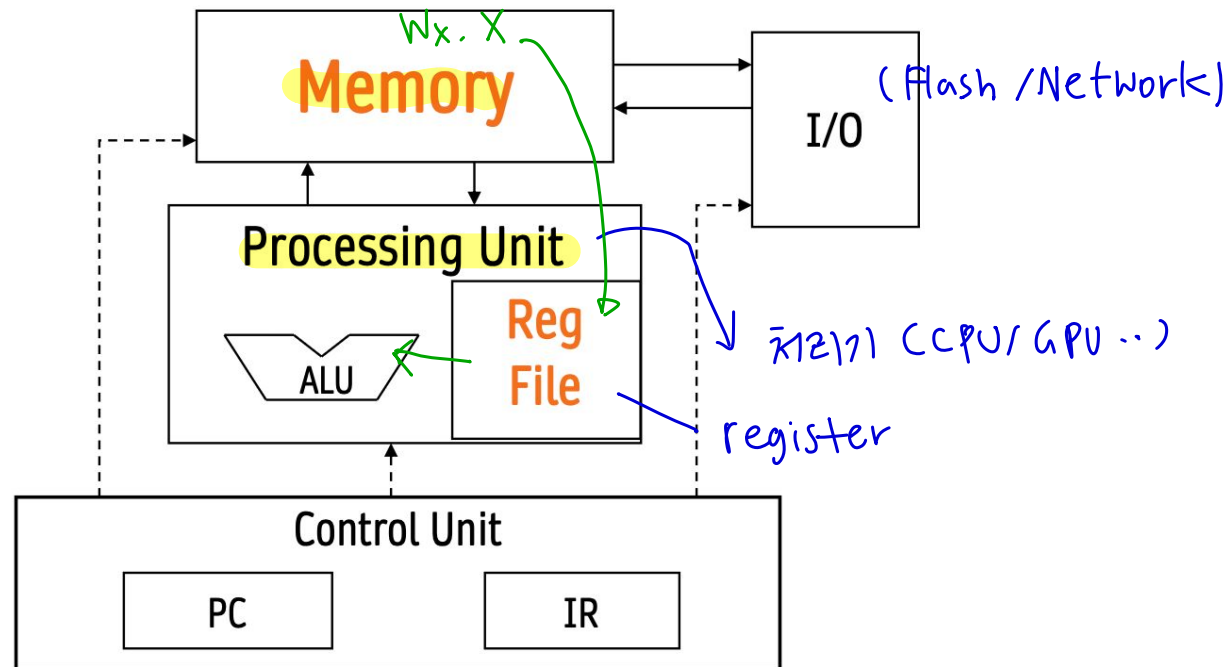
- Less flexible pruning index choice (a subset of the fine-grained case)
- Easy to accelerate (just a smaller matrix!)

■ Preserved
□ Pruned



Computation Requires Moving Data

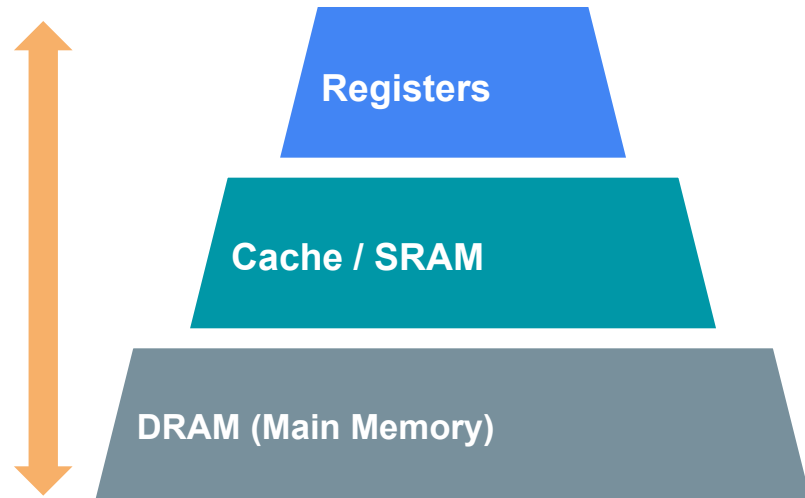
- Arithmetic happens on values in *registers*, not memory.
- Even a simple operation requires memory load/store.
- For ML operators, moving tensor data can cost more than arithmetic.



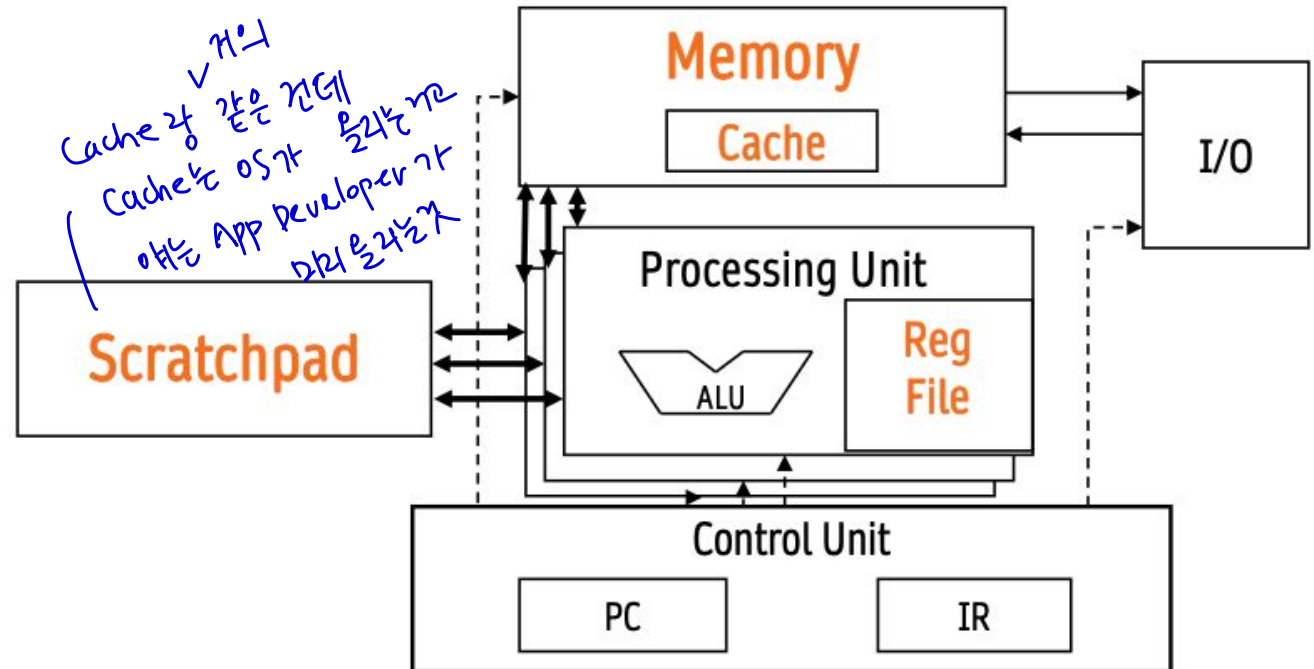
Memory Hierarchy: Fast Memory is Small

- Register is small but fast; DRAM is large but slow.
- Cache (or SRAM/scratchpad) resides between registers and DRAM.
- Cache reduces DRAM access by keeping nearby or recently used data close.

Faster • Smaller

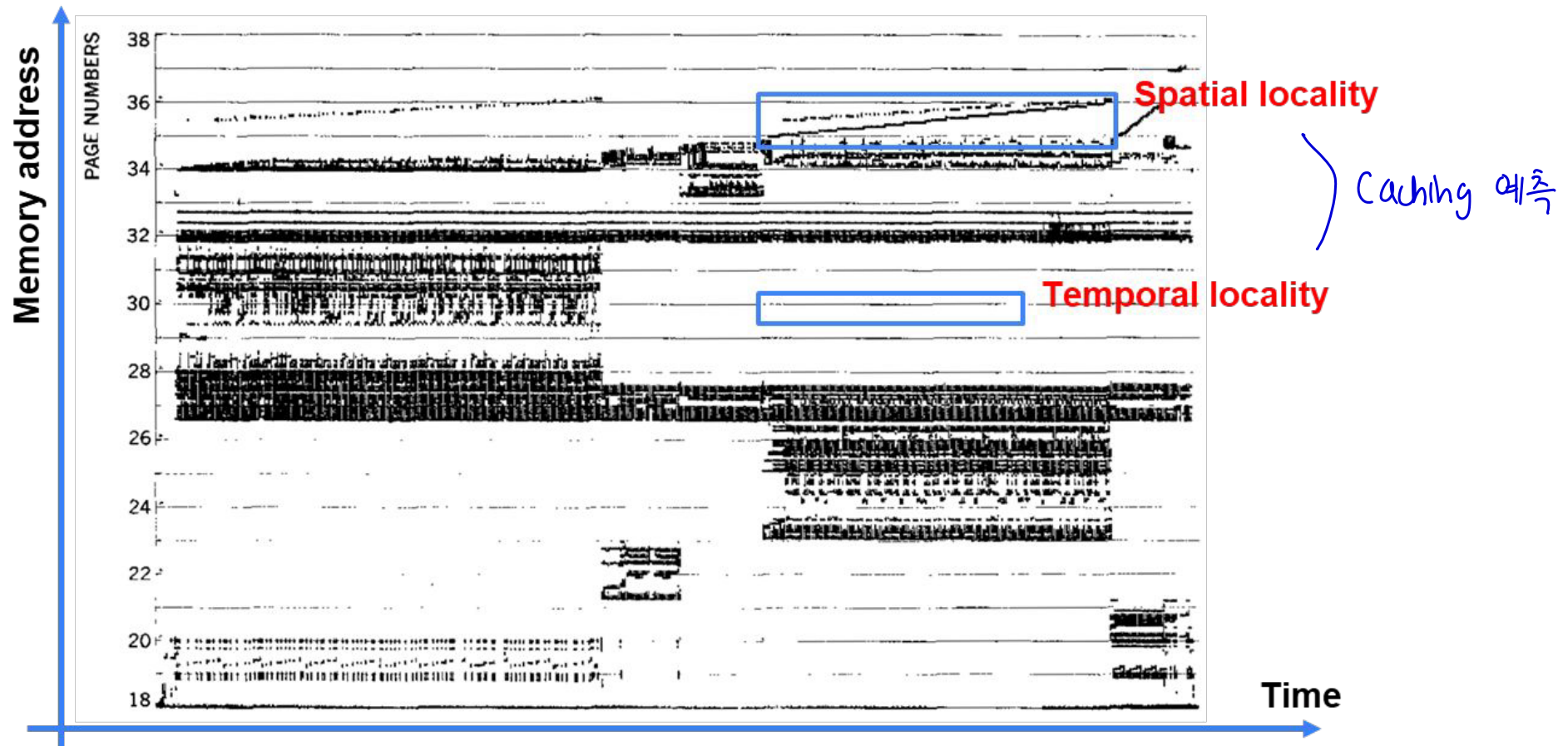


Slower • Larger



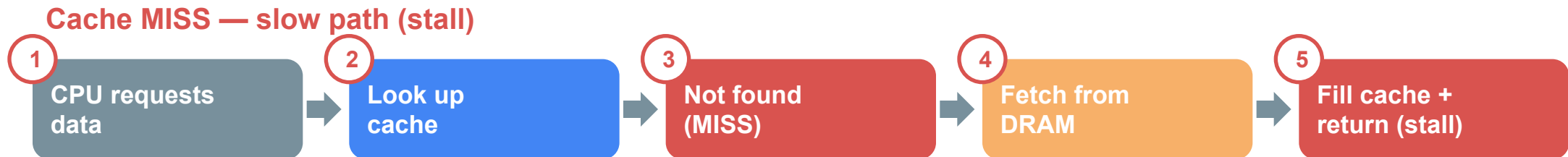
Temporal Locality and Spatial Locality

- Temporal locality: recently used data is reused soon.
- Spatial locality: nearby addresses are accessed together.



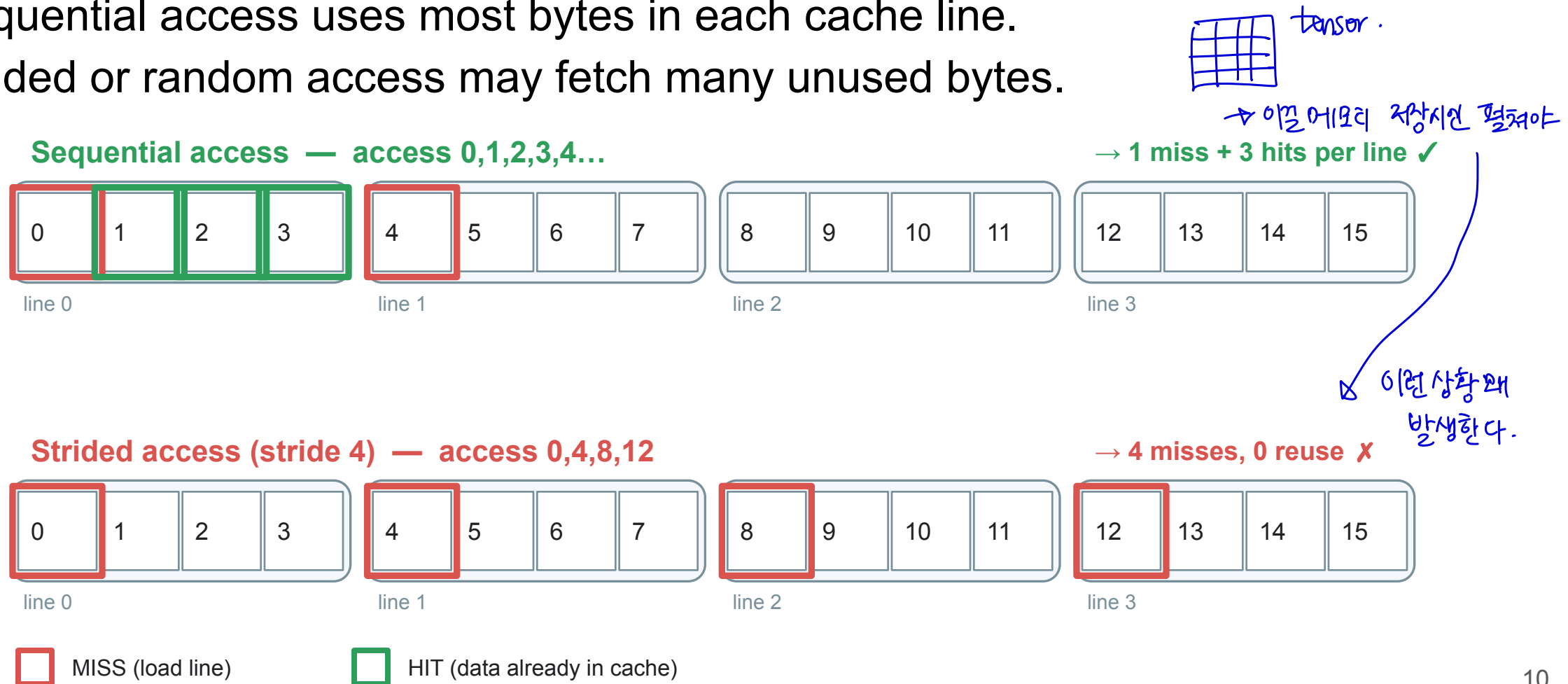
Cache Hit, Cache Miss, and Stalls

- Cache **hit**: data is found close to the compute unit, so loads are quick.
- Cache **miss**: data must be fetched from a lower level, stalling execution.
- Misses reduce utilization because compute units wait for data.



Cache Make Access Patterns Matter

- Data moves between DRAM and cache in cache-line-sized blocks.
- Sequential access uses most bytes in each cache line.
- Strided or random access may fetch many unused bytes.



Data Movement Can Dominate ML Latency

- FLOPs ignore where data comes from and how often it moves.
- Cache behavior and memory bandwidth can dominate operator latency.
- Efficient ML requires fewer bytes moved, better locality, and higher reuse.

Table 1. Proportions for operator classes in PyTorch.

Operator class	% flop	% Runtime
△ Tensor contraction	99.80	61.0
□ Stat. normalization	0.17	25.5
○ Element-wise	0.03	13.5

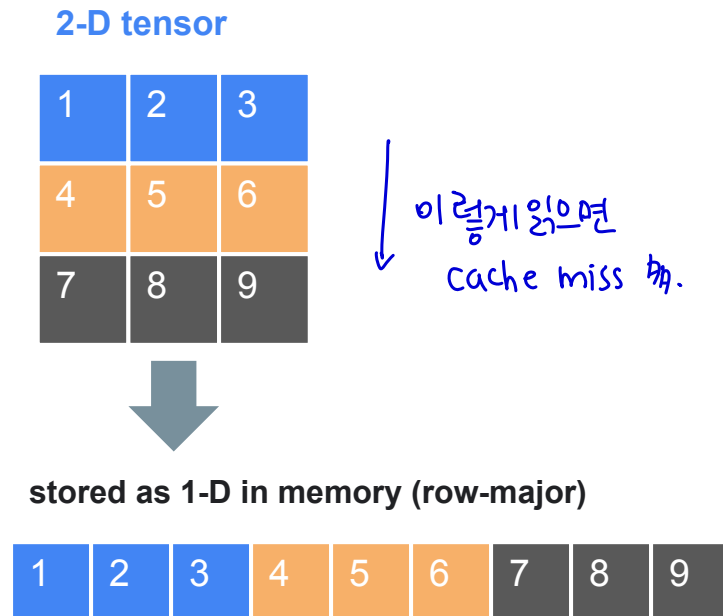
→ cachehit을 위한 최적화 해 done.

0.2% 30% → 아직 yet.

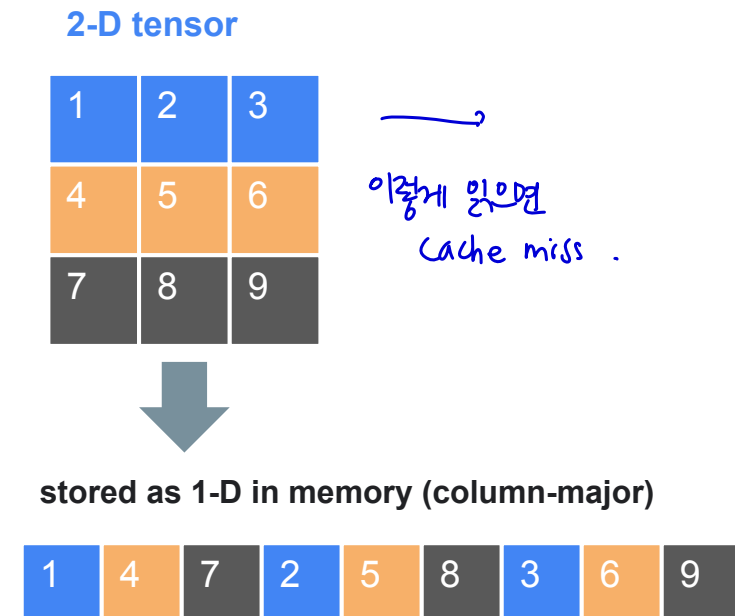
Runtime breakdown of a BERT model training

Tensor Layout Determines Access Patterns

- Tensors appear multidimensional, but they are stored as linear arrays.
- How data is stored in a linear array significantly affects the cache miss ratio.

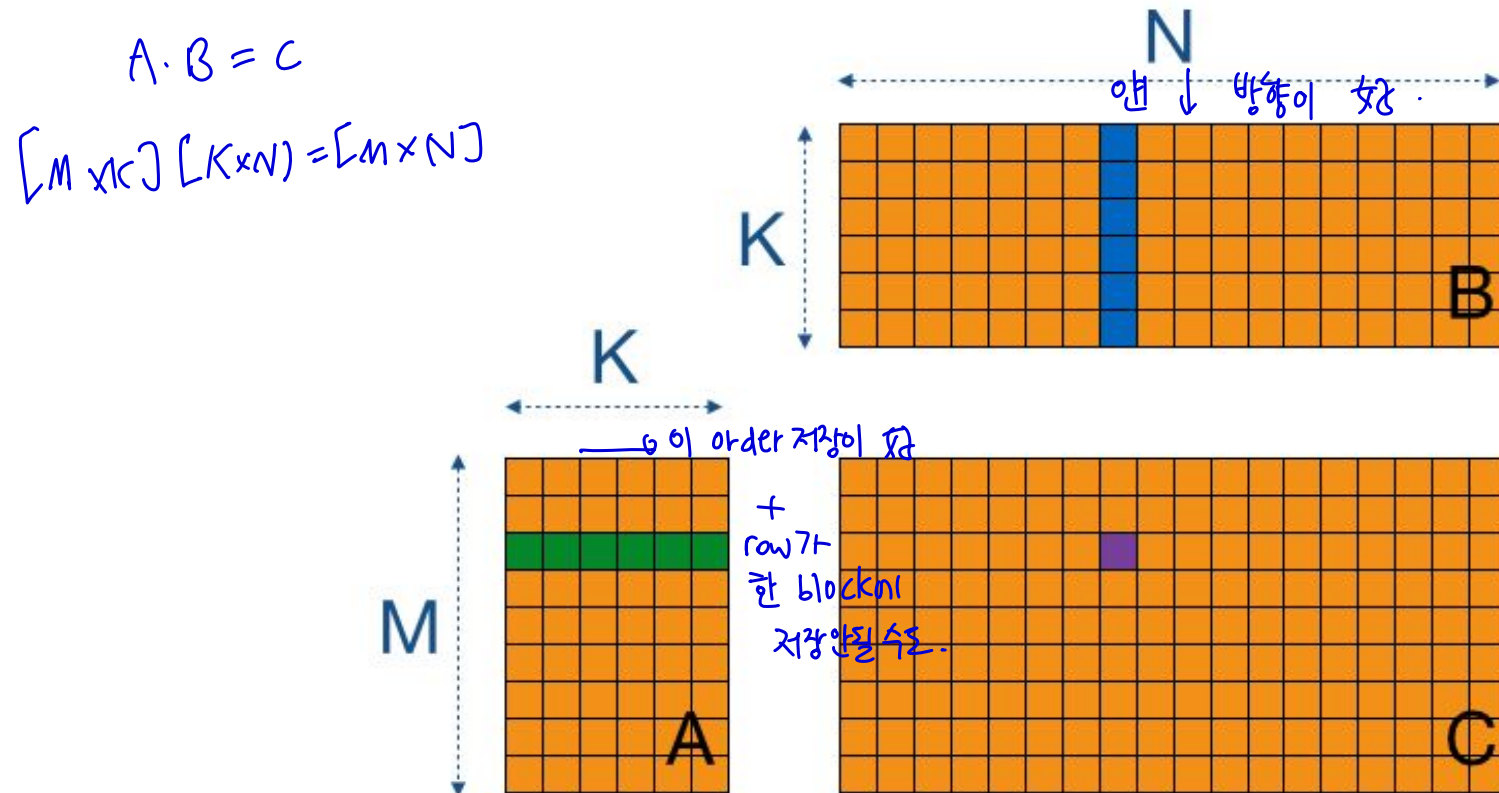


이런
패턴에 맞춘
저장해야 한다.



General Multiplication (Library) Naïve GEMM Wastes Locality

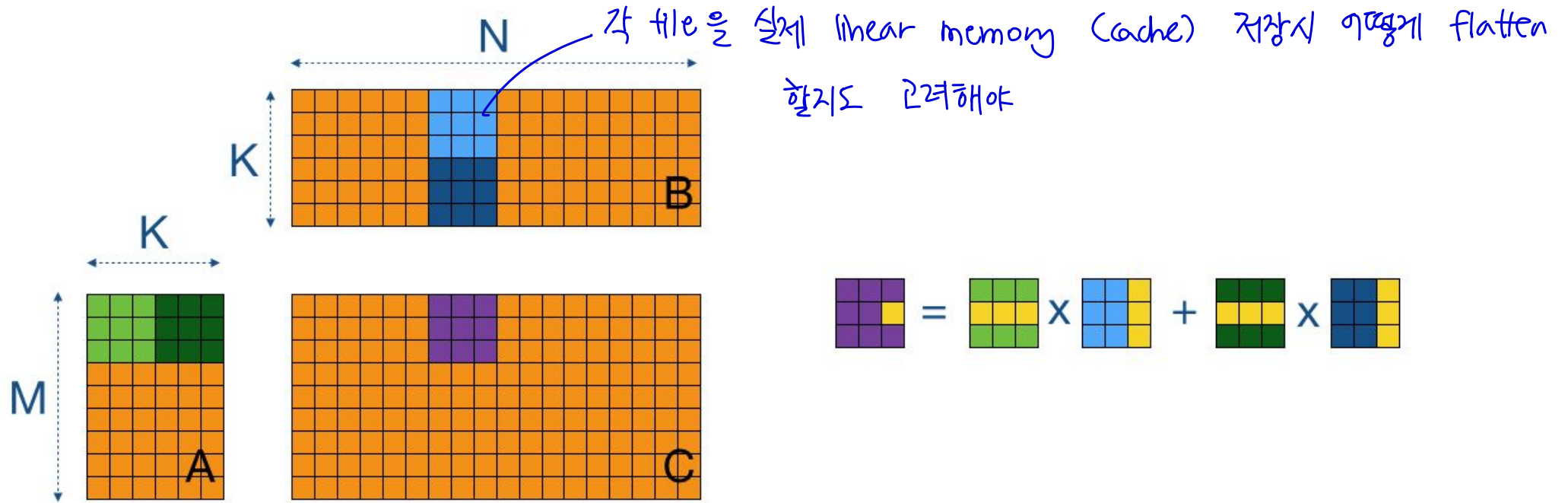
- Simple loops may create poor locality depending on loop order.
- Column-wise traversal is inefficient on row-major layouts.
- The same FLOP count can produce very different runtimes.



Tiling Increases Reuse

1 Tile $\approx$ cache block size

- Tiling partitions a large tensor operation into smaller blocks that fit in cache.
- While a tile remains in fast memory, it is reused across many computations.
- The goal is to reduce DRAM traffic and increase reuse.

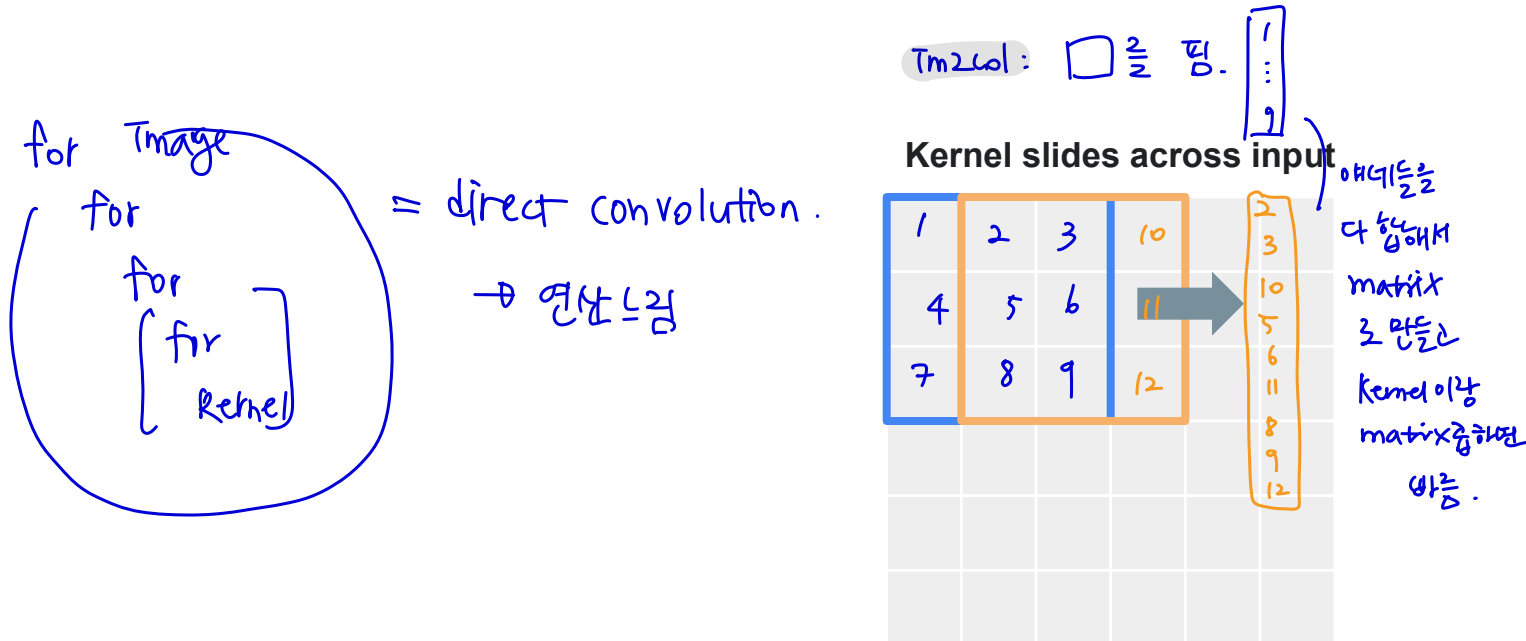


$$\begin{bmatrix} \text{purple} & \text{yellow} \\ \text{purple} & \text{yellow} \end{bmatrix} = \begin{bmatrix} \text{green} & \text{yellow} \\ \text{green} & \text{yellow} \end{bmatrix} \times \begin{bmatrix} \text{blue} & \text{yellow} \\ \text{blue} & \text{yellow} \end{bmatrix} + \begin{bmatrix} \text{dark green} & \text{yellow} \\ \text{dark green} & \text{yellow} \end{bmatrix} \times \begin{bmatrix} \text{dark blue} & \text{yellow} \\ \text{dark blue} & \text{yellow} \end{bmatrix}$$

$$\begin{bmatrix} \text{purple} & \text{yellow} \\ \text{purple} & \text{yellow} \end{bmatrix} = \begin{bmatrix} \text{green} & \text{yellow} \\ \text{green} & \text{yellow} \end{bmatrix} \times \begin{bmatrix} \text{blue} & \text{yellow} \\ \text{blue} & \text{yellow} \end{bmatrix} + \begin{bmatrix} \text{dark green} & \text{yellow} \\ \text{dark green} & \text{yellow} \end{bmatrix} \times \begin{bmatrix} \text{dark blue} & \text{yellow} \\ \text{dark blue} & \text{yellow} \end{bmatrix}$$

Data Reuse In Convolution

- Each convolution weight is reused across many spatial positions.
- Input activations are reused because adjacent pixels have overlapping fields.
- Convolution performance depends on how well this reuse maps.



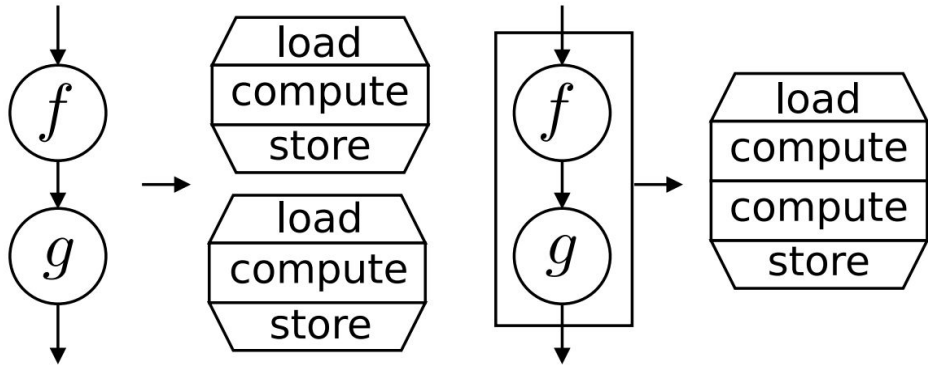
Overlapping windows reuse the same input pixels many times

Elementwise Operators Have Low Reuse

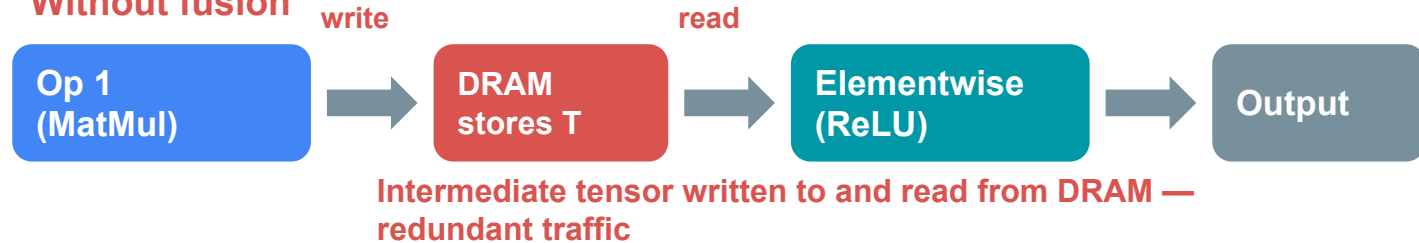
(서로 연관성X) $\rightarrow$ 데이터를 한번 읽으면 개별 데이터별로 연산하는 것 : ex) 선형연산 / activation $\rightarrow$ 의존하는 것에 영향이 (필요성↓).

$\circ$ 배치크기가 크면 OK

- $C[i] = f(A[i], B[i])$ usually reads and writes each element once.
- These operators perform little arithmetic per byte moved.
- Fusion improves performance by eliminating intermediate tensor movements.



Without fusion



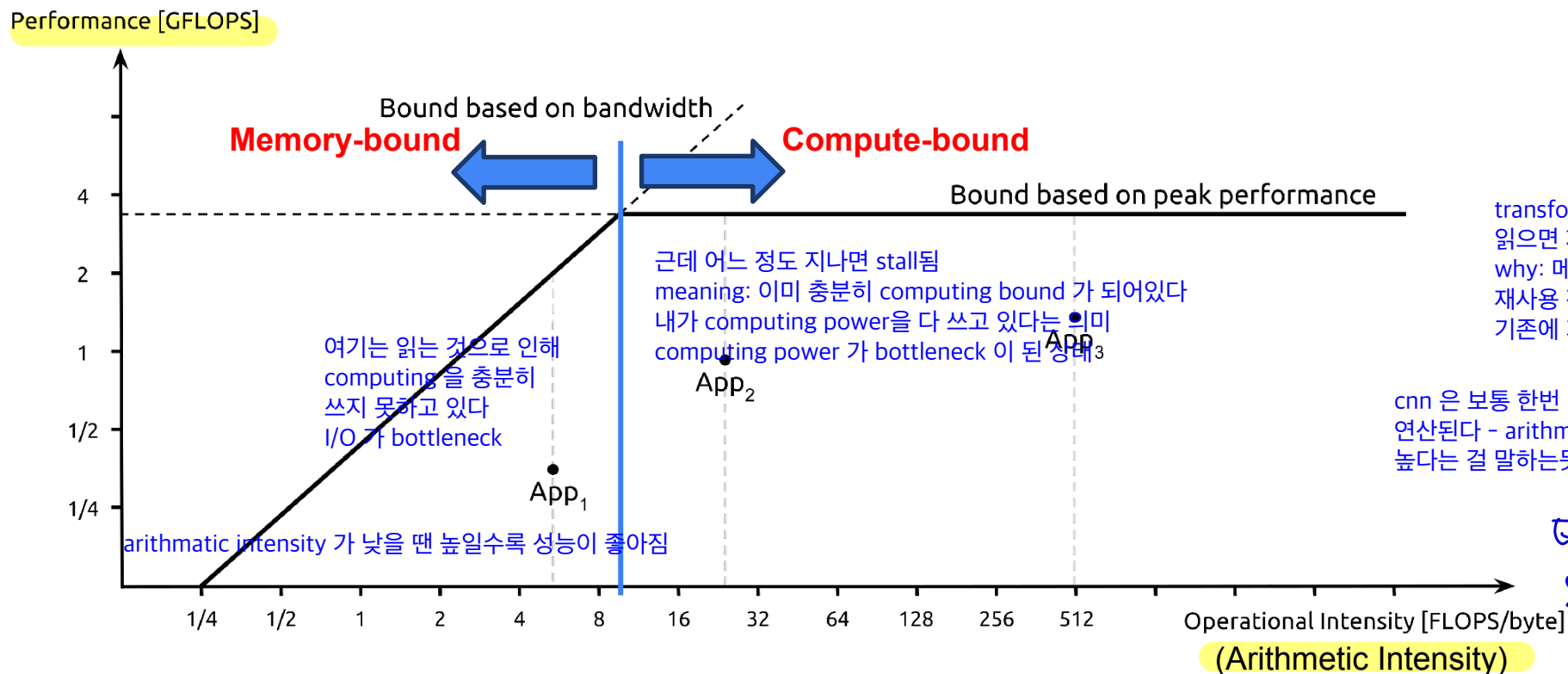
With fusion



No intermediate DRAM write/read — value stays in registers / on-chip

Arithmetic Intensity and Roofline Model

- Arithmetic intensity is the number of operations performed per byte moved.
- Low-intensity operators tend to be memory-bound.
- High-intensity operators tend to be compute-bound.



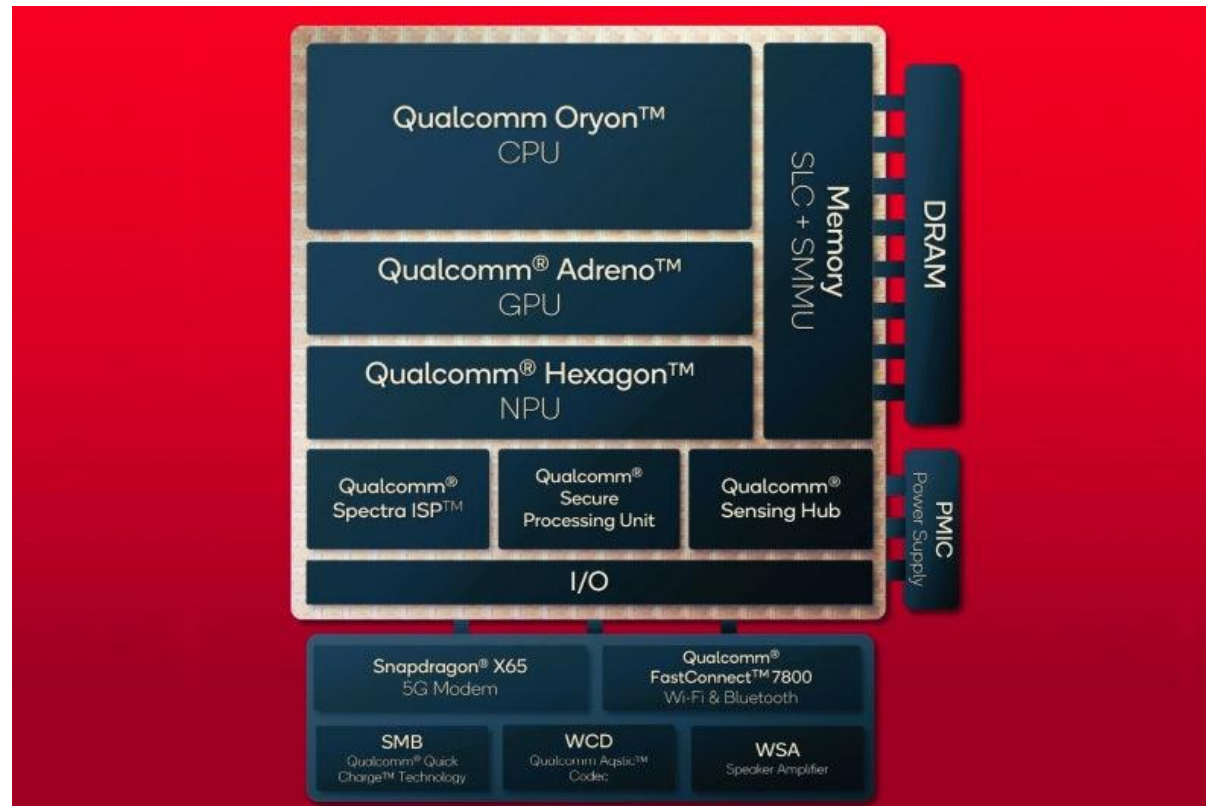
transformer는 weight를 한번 읽으면 재사용이 어렵다
why: 메모리 사이즈 제약때문에 재사용 전에 다른 계산을 많이 해야해서 기존에 가져온거 날려야함

cnn은 보통 한번 읽으면 여러번 연산된다 - arithmetic intensity가 높다는 걸 말하는듯

데이터를 메모리에서 읽어서 연산을 했거나
한번만

Parallel Execution Matters for ML

- Neural network operators expose large amounts of data parallelism.
- Mobile SoC is equipped with heterogeneous processors.
- CPU, GPU, and NPU have different parallelism models.

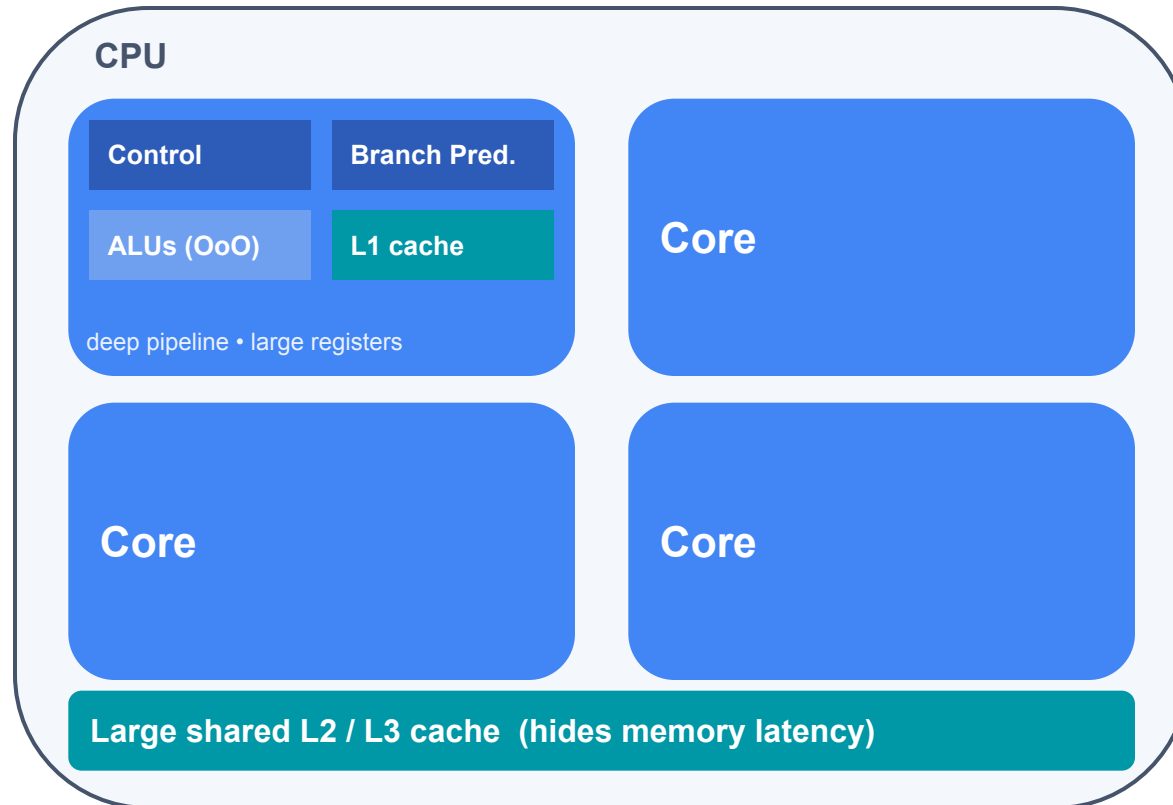


GPU/NPU 는 특수 연산에 특화된
차이:

- GPU: 매트릭스 연산 특화, 학습에 많이 쓰임 - 학습은 지원해야하는 연산이 더 많음. GPU 는 좀더 다양한 연산 기능 지원. 그리고 계산에 floating point 계산을 해야함
- NPU: 파워 소모를 줄이기 위해, 추론은 제한적인 연산만으로 가능하기 때문에 이걸 쓴다. 추론은 int 나 fp16 정도만으로 되기 때문에 작은 type 연산만 가능해도 됨
- NPU 가 GPU 보다 빠르고 전력 소모가 낮다.
- 최신 모델은 GPU 에서만 가능할수도 있음

CPU: Latency-Oriented General-Purpose Cores

- CPUs use a small number of cores optimized for low-latency execution.
- They handle control flow, preprocessing, and irregular operators well.
- ML performance on CPUs depends on SIMD and thread scheduling.



CPU Parallelism: Threads and SIMD

→ 2가지

①

②

→ 동일 thread 내에서도

1. SIMD — data parallelism (one core)

A a0 a1 a2 a3

B b0 b1 b2 b3

배열 A, B 에서 하나씩 받아서 더하고 싶다
A, B 에서 읽고 각각 더하는 건 병렬처리 할 수도 있다

1 instruction: VADD
(applied to all lanes)

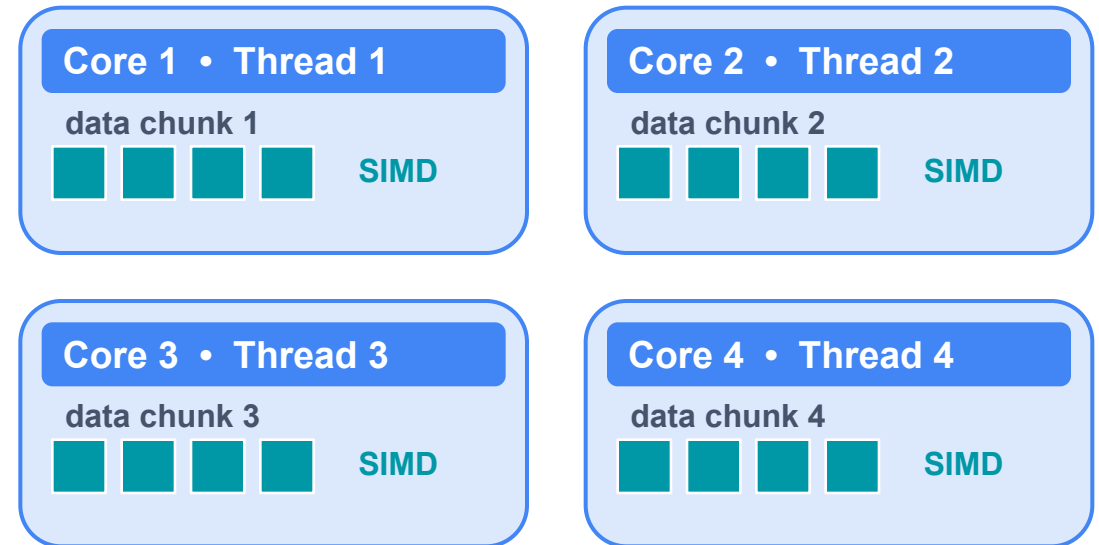
C a0+b0 a1+b1 a2+b2 a3+b3

One instruction processes 4 data elements at once
(vector width = 4)

2. Threads — task parallelism (many cores)

서로 다른 작업 단위.

개별 core 별 다른 작업시킨다.



Independent thread per core; SIMD inside each core

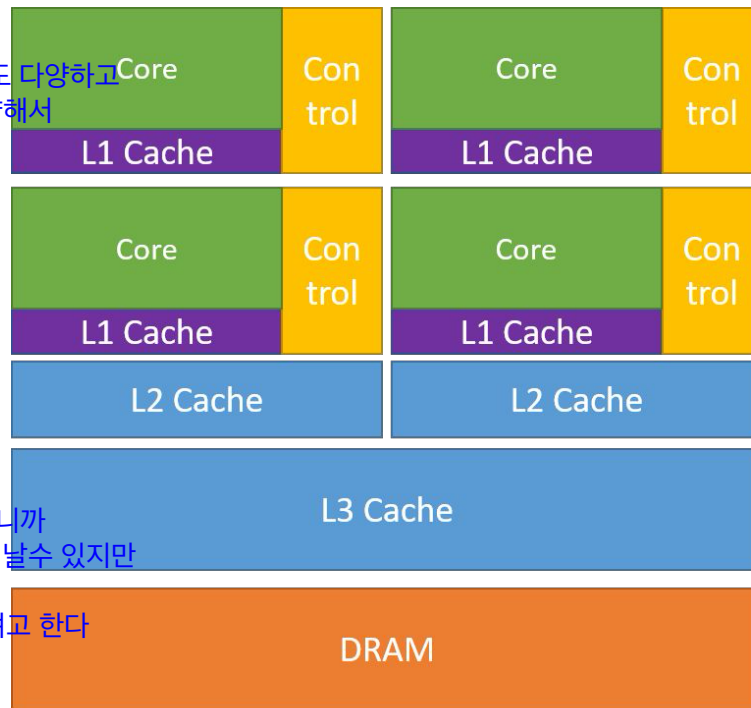
CPUs combine both: a few threads (cores) × SIMD lanes per core

GPU: Throughput-Oriented Massive Parallelism

- GPUs use many lightweight execution lanes to maximize throughput.
- Dense tensor operations perform well when there are enough independent elements to keep the GPU occupied.

지원해야하는 데이터 타입도 다양하고
해야하는 operation도 다양해서
많이 두기 어려움

연산이 다양하고, task가 많으니까
cache를 하나만 두면 miss가 날수 있지만
core 별로 다른 일을 하니까
각각 caching 정책을 따로 두려고 한다



CPU

Control units are shared across multiple cores



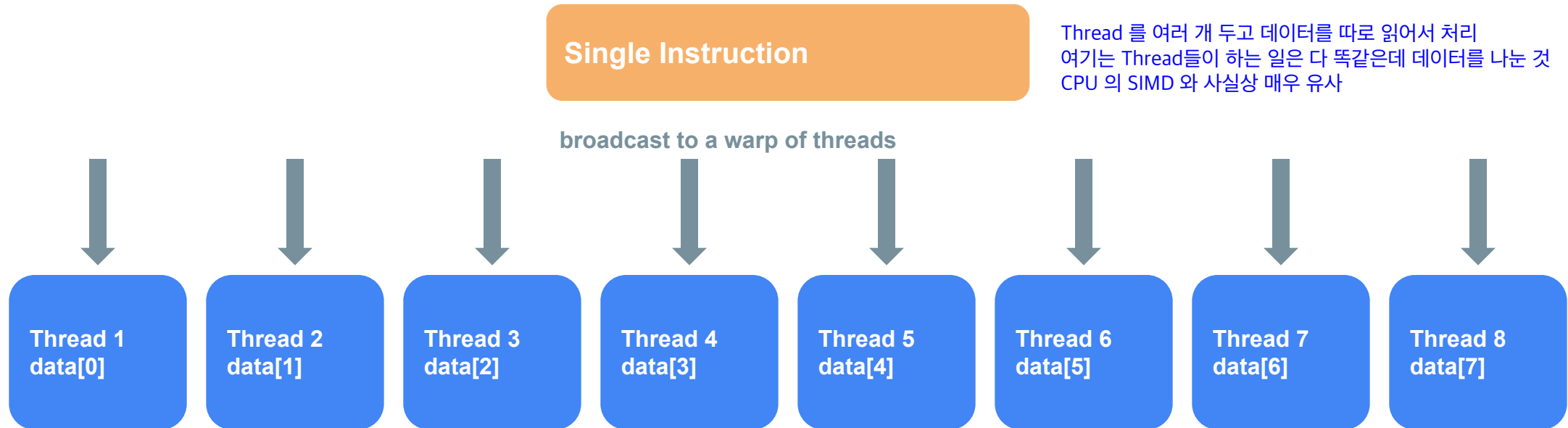
Core가 엄청 많음
GPU는 덧셈 곱셈만 하니까
많이 둘 수 있음

Cache가 하나밖에 없다
왜? 데이터를 미리 fetch 해야하는
방식이 하나뿐임. 그래서 하나만 뒤도 됨

GPU

GPU Parallelism: SIMT

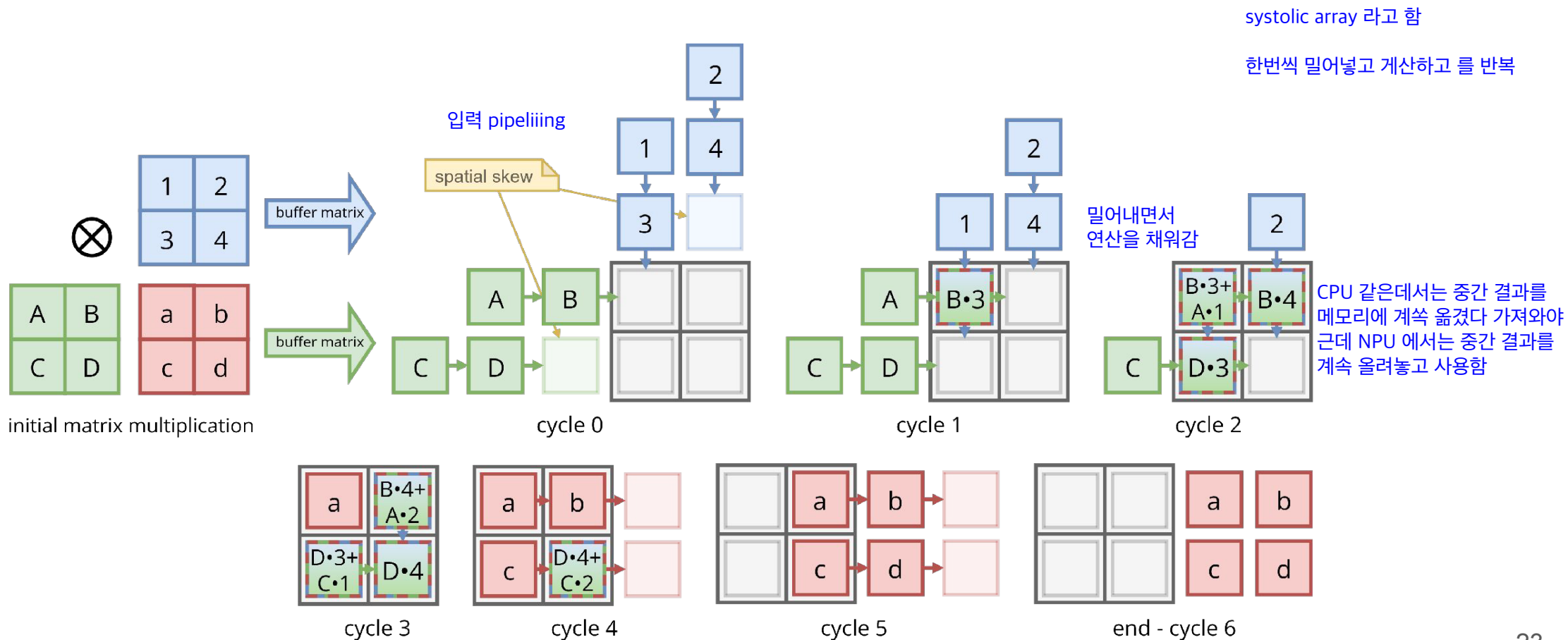
- **S**ingle **I**nstruction, **M**ultiple **T**hreads
- Each thread executes the same code but operates on a different piece of data.
- Each thread has its own context.



SIMT: all threads run the same instruction on different data

NPU/TPU: Dataflow-Oriented Execution

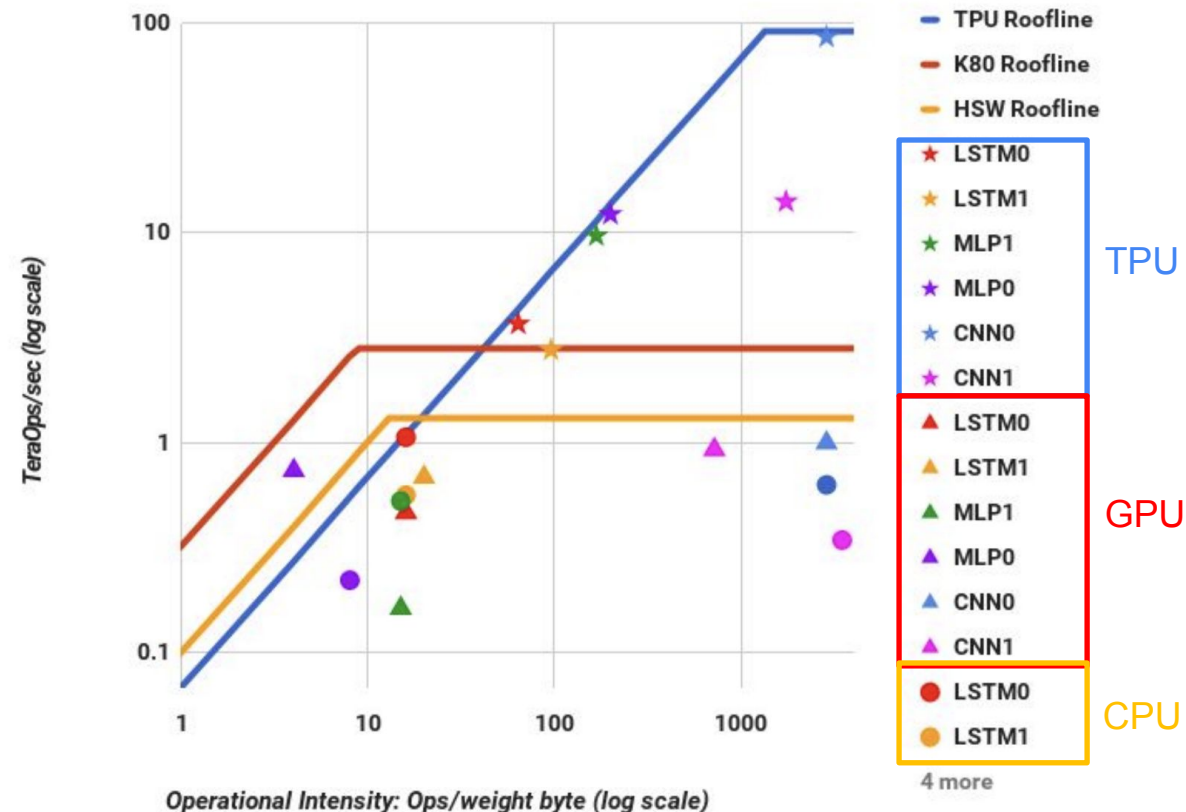
- NPUs specialize common neural network kernels using MAC arrays.
- Performance depends on whether the operator maps to supported dataflows.



Comparison across Processors

- Different hardware, different theoretical bounds.
- Different operators, different performance/efficiency.

Log-Log Scale



Summary

- In this lecture, we learned:
 - Why systems matter for ML?
 - Memory hierarchy and data movement
 - Tensor computation as data reuse
 - Parallel execution models